Beispiele

- Suchen und Sortieren:
  - Binäre Suche in $O(\log n)$.
  - Median eines Felders in $O(n)$.
  - Quicksort in $O(n \log n)$ im durchschnittlichen Fall.
  - Mergesort in $O(n \log n)$.
- Algebraische Algorithmen
  - Multiplikation von Matrizen in $O(n^{2.8})$.
  - FFT (Fast Fourier Transformation) in $O(n \log n)$.
  - Multiplikation von Polynomen in $O(n \log n)$.
  - Arithmetische Operationen mit großen, ganzen Zahlen (mehrere 100 Stellen).

§2.2 Laufzeit von Teile-und-Herrsche-Algorithmen

- Ein Teile-und-Herrsche-Algorithmus mit Laufzeitfunktion $T(n)$

Basisfall	1.	Löst den Basisfall mit Laufzeit $f_B(n)$,
Teilen	2.	zerlegt ein Problem der Größe n mit Laufzeit $f_T(n)$ in $a_0 \geq 0$ Teilprobleme der Größe n/b mit $b > 1$,
Rekursion	3.	löst $a \leq a_0$ der Teilprobleme rekursiv mit Laufzeit $T(n/b)$ und
Herrschen	4.	fügt die a Teillösungen mit Laufzeit $f_H(n)$ zusammen.

f heißt Split-Funktion

  - Für den Basisfall (Schritt 1.), das Teilen (Schritt 2.) und Herrschen (Schritt 4.) wird insgesamt Laufzeit $f = f_B + f_T + f_H$ aufgewendet.

- Die gesamte Laufzeit $T(n) = a \cdot T(n/b) + f(n)$ eines Teil-Und-Herrsche-Algorithmus kann dann mit dem

  Master-Theorem

  berechnet werden.

§2.2 Laufzeit von Teile-und-Herrsche-Algorithmen

Theorem (Master Theorem für Teile-und-Herrsche)

Sei $T(n)$ eine Laufzeitfunktion der Form $T(n) = a \cdot T(n/b) + f(n)$ mit

- Konstanten $a \geq 1$, $b > 1$ und $\alpha := \log_b a$ (aka watershed constant),
- einer Funktion (aka splitting function) $f(n) > 0$ für hinreichend großes n, und
- n/b interpretiert als $\lceil n/b \rceil$ oder $\lfloor n/b \rfloor$.

Dann gelten für $T(n)$ die folgenden Abschätzungen:

1. …
2. …
3. …

Achtung: Für $b \leq 1$ liegt kein T&H-Algorithmus vor und das Master-Theorem ist nicht anwendbar!

Theorem (Master Theorem für Teile-und-Herrsche)

Sei $T(n)$ eine Laufzeitfunktion der Form $T(n) = a \cdot T(n/b) + f(n)$ mit …

Dann gelten für $T(n)$ die folgenden Abschätzungen:

1. Falls $f(n) = O(n^{\alpha-\varepsilon})$ für $\varepsilon > 0$, dann ist $T(n) = \Theta(n^{\alpha})$.
2. Falls $f(n) = \Theta\left(n^{\alpha} \log^{\delta} n\right)$
   - a0) mit $\delta = 0$, dann ist $T(n) = \Theta(n^{\alpha} \log n)$,
   - a1) mit $\delta > -1$ (insb. Standardfall $\delta = 0$), dann ist $T(n) = \Theta\left(n^{\alpha} \log^{\delta+1} n\right)$,
   - b) mit $\delta = -1$, dann ist $T(n) = \Theta(n^{\alpha} \log \log n)$,
   - c) mit $\delta < -1$, dann ist $T(n) = \Theta(n^{\alpha})$.
3. Falls $f(n) = \Omega(n^{\alpha+\varepsilon})$ für $\varepsilon > 0$, dann ist $T(n) = \Theta(f(n))$,

   falls zusätzlich $a \cdot f(n/b) \leq c \cdot f(n)$ für $c < 1$ und hinreichend großes n.

Bemerkungen zum Master-Theorem für Teile-und-Herrsche

Fall 1: Der Aufwand für Teilen/Herrschen/Basisfälle **wird dominiert** durch den Aufwand zur Lösung der Teilprobleme.

- Das Hauptgewicht des Rekursionsbaums liegt in den Blättern (aka leaf-heavy).

Fall 2: Der Aufwand für Teilen/Herrschen/Basisfälle **ist asymptotisch ausbalanciert** mit dem Aufwand zur Lösung der Teilprobleme.

Fall 3: Der Aufwand für Teilen/Herrschen/Basisfälle **dominiert** den Aufwand zur Lösung der Teilprobleme.

- Das Hauptgewicht des Rekursionsbaums liegt in der Wurzel (aka root-heavy).

Die drei Fälle sind nicht erschöpfend, d.h. es gibt nicht abgedeckte Fälle

z.B. $f(n) = n^{\alpha} \cdot \log n \cdot \log\log n$ oder $f(n) = n^{\alpha} \cdot (\log\log n)^{r}$.

- Allgemeiner Versionen des Theorems werden benötigt und gibt es auch…
- … oder Substitutionen verwenden.

§2.2 Laufzeit von Teile-und-Herrsche-Algorithmen

```
static int search(int[] arr, int li, int re, int key)
{  // Voraussetzung: a ist aufsteigend sortiert

   if (re-li <      0) return -1;
   int m = (li+re)/2;
   if (key == arr[m]) return m;
   if (key <  arr[m]) return search(arr, li,m-1,key);
   else               return search(arr,m+1, re,key);
}
```

- Worst-case-Laufzeit der rekursiven Suche:
  - Annahme: Das Feld ist aufsteigend sortiert.
  - Laufzeitfunktion $T(n) = 1 \cdot T(n/2) + 10$
  - Master-Theorem: $a = 1, b = 2, \delta = 0, \alpha = 0, f(n) = O(1)$
  - Fall 2a: $T(n) = \Theta(\log n)$

§2.2a Laufzeit von rekursiven Algorithmen

Theorem (Master Theorem für subtract-and-conquer)

Sei $T(n)$ eine Laufzeitfunktion der Form $T(n) = a \cdot T(n-b) + f(n)$ mit

- Konstanten $a > 0$ und $b > 0$ sowie
- einer Funktion $f(n) \in O(n^d)$, $d \geq 0$.

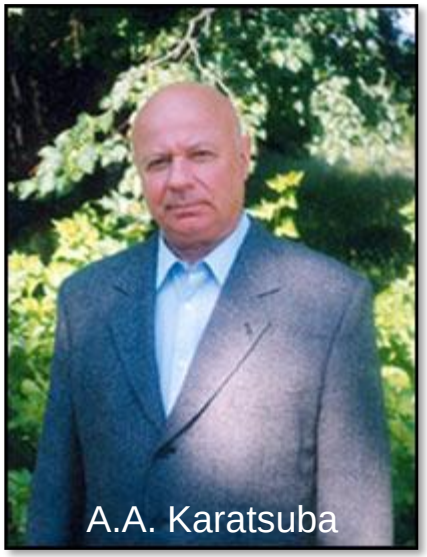
Dann gelten für $T(n)$ die folgenden Abschätzungen:

1. Falls $a < 1$, dann ist $T(n) = O(n^d)$.
2. Falls $a = 1$, dann ist $T(n) = O(n^{d+1})$.
3. Falls $a > 1$, dann ist $T(n) = O(n^d \cdot a^{n/b})$.

Achtung: Wenn dieses Theorem anwendbar ist, dann handelt es sich bei dem Algorithmus um **KEINEN** Teile-und-Herrsche Algorithmus!

§2.3 Der Algorithmus von Karatsuba

Problem: Multiplikation zweier Zahlen u und v der Länge n.

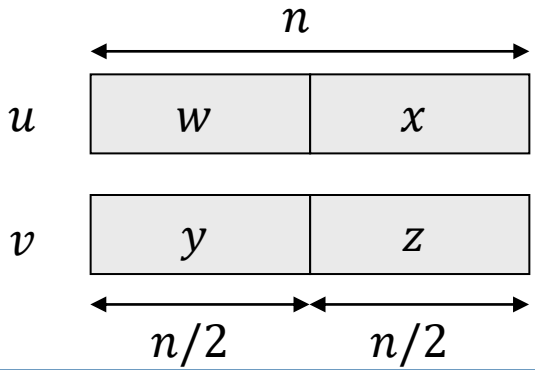

A.A. Karatsuba

Naiver Algorithmus:

- „Schul“-Algorithmus: $T(n) = O(n^2)$.

Idee:

- Teile-und-Herrsche-Ansatz [1962]:
  - Zerlege beide Zahlen in Zahlen der halben Länge.



$$uv = (w \cdot 10^{n/2} + x) \cdot (y \cdot 10^{n/2} + z)$$
$$= wy \cdot 10^{n} + (wz + xy) \cdot 10^{n/2} + xz$$

Direkte Umsetzung der Idee

- Die Multiplikation zweier Zahlen u und v der Länge n wird zurückgeführt auf **vier** Multiplikationen von Zahlen der Länge $n/2$:

$$uv = wy \cdot 10^n + (wz + xy) \cdot 10^{n/2} + xz.$$

- Zusätzlich kommt ein linearer Aufwand für drei Additionen und Multiplikation mit 10^n und $10^{n/2}$ (Links-Shift-Operation) dazu, also

$$T(n) = 4 \cdot T(n/2) + O(n).$$

- Master-Theorem: $a = 4, b = 2, f(n) = O(n)$
- Fall 1:

$$T(n) = \Theta(n^{\log_2 4}) = O(n^2).$$

- Keine Verbesserung gegenüber dem naiven Algorithmus.
- Warum? Zu viele Teil-Multiplikationen, d.h. verkleinere a.

§2.3 Der Algorithmus von Karatsuba

Verbesserung:

- Betrachte das Produkt:

$$(w + x) \cdot (y + z) = wy + (wz + xy) + xz.$$

- In diesem Ausdruck kommen alle Produkte vor, die auch in uv berechnet werden müssen:

$$uv = wy \cdot 10^n + (wz + xy) \cdot 10^{n/2} + xz.$$

- Damit lässt sich uv mit drei Multiplikationen berechnen:

$$\begin{aligned} r &= (w + x) \cdot (y + z); \\ p &= w \cdot y; \\ q &= x \cdot z; \\ uv &= p \cdot 10^n + (r - p - q) \cdot 10^{n/2} + q; \end{aligned}$$

§2.3 Der Algorithmus von Karatsuba

Verbesserung:

- Zusätzlich kommt ein linearer Aufwand für sechs Additionen und Multiplikation mit 10^n und $10^{n/2}$ (Links-Shift-Operation) dazu, also

$$T(n) = 3 \cdot T(n/2) + O(n).$$

- Master-Theorem: $a = 3,\ b = 2,\ f(n) = O(n).$
- Fall 1:

$$T(n) = \Theta(n^{\log_2 3}) = O(n^{1.58}).$$

§2.3 Der Algorithmus von Karatsuba

Bemerkungen

- Toom-Cook [1963/66]:

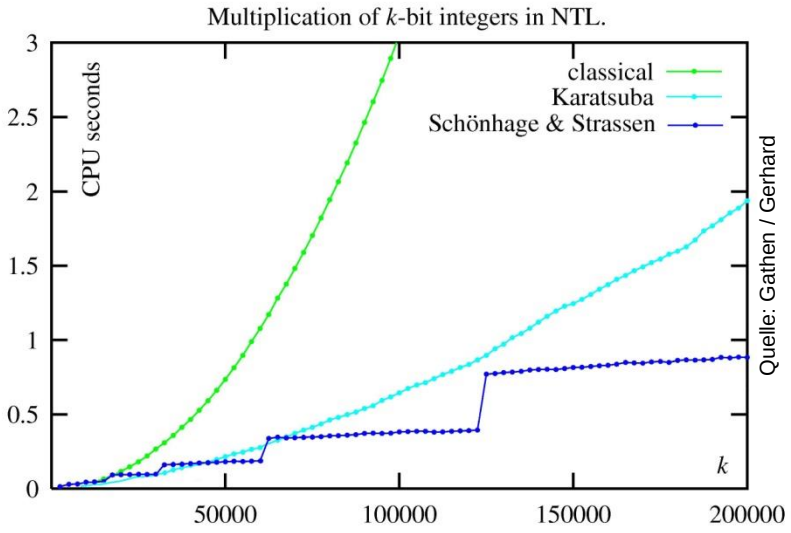
$$O\left(n \log(n)\, 2^{\sqrt{2 \log n}}\right).$$

- Schönhagen-Strassen [1971]:

$$O(n \log(n) \log(\log n)).$$

- Fürer [2007]: $O\left(n \log(n)\, 2^{O(\log^\star n)}\right)$.
  - $\log^\star n = \min\{i \geq 0 : \log^{(i)} n \leq 1\}$

    mit $\log^{(0)} n = n$ und $\log^{(i+1)} n = \log \log^{(i)} n$



Quelle: Gathen / Gerhard

- D. Harvey, J. van der Hoeven [2021]: **Integer multiplication in time $O(n \log n)$**, Annals of Mathematics, 193 (2): 563-617, March 2021.

§2.4 Der Algorithmus von Strassen

Problem: Multiplikation zweier $n \times n$ Matrizen A und B:

$$C = A \cdot B.$$



Naiver Algorithmus:

- „Schulbuch“-Algorithmus: $T(n) = O(n^3)$.

```
for (int i=0; i<n; i++)
   for (int j=0; j<n; j++)
   {
      C[i][j] = 0;
      for (int k=0; k<n; k++)
         C[i][j] += A[i][k] * B[k][j];
   }
```

§2.4 Der Algorithmus von Strassen

- Erst im Jahr 1968 wurde die untere Schranke $\Omega(n^3)$ von Volker Strassen durchbrochen.

Idee:

- Teile-und-Herrsche-Ansatz [1968]:
- Zerlege beide Matrizen in vier Teilmatrizen der Größe $\frac{n}{2} \times \frac{n}{2}$:

$$\begin{bmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{bmatrix} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix} \cdot \begin{bmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{bmatrix}.$$

- Dann gilt:

$$C_{1,1} = A_{1,1} \cdot B_{1,1} + A_{1,2} \cdot B_{2,1},$$
$$C_{1,2} = A_{1,1} \cdot B_{1,2} + A_{1,2} \cdot B_{2,2},$$
$$C_{2,1} = A_{2,1} \cdot B_{1,1} + A_{2,2} \cdot B_{2,1},$$
$$C_{2,2} = A_{2,1} \cdot B_{1,2} + A_{2,2} \cdot B_{2,2}.$$

§2.4 Der Algorithmus von Strassen

- Eine Matrixmultiplikation mit diesen Regeln benötigt dann
  - **acht** Matrixmultiplikationen der Größe $\left(\frac{n}{2} \times \frac{n}{2}\right)$ und
  - vier Matrixadditionen der Größe $\left(\frac{n}{2} \times \frac{n}{2}\right)$ mit Laufzeit $O(n^2)$:

$$T(n) = 8 \cdot T(n/2) + O(n^2).$$

- Master-Theorem: $a = 8, b = 2, f(n) = O(n^2)$.
- Fall 1:

$$T(n) = \Theta\left(n^{\log_2 8}\right) = O(n^3).$$

- Warum?
  - Zu viele Teil-Multiplikationen, d.h. verkleinere a.

§2.4 Der Algorithmus von Strassen

- Man kommt auch mit **sieben** Multiplikationen aus:

$$M_1 = (A_{1,2} - A_{2,2}) \cdot (B_{2,1} + B_{2,2}),$$
$$M_2 = (A_{1,1} + A_{2,2}) \cdot (B_{1,1} + B_{2,2}),$$
$$M_3 = (A_{1,1} - A_{2,1}) \cdot (B_{1,1} + B_{1,2}),$$
$$M_4 = (A_{1,1} + A_{1,2}) \cdot B_{2,2},$$
$$M_5 = A_{1,1} \cdot (B_{1,2} - B_{2,2}),$$
$$M_6 = A_{2,2} \cdot (B_{2,1} - B_{1,1}),$$
$$M_7 = (A_{2,1} + A_{2,2}) \cdot B_{1,1}.$$

- Mit acht weiteren Additionen ergibt sich C:

$$C_{1,1} = M_1 + M_2 - M_4 + M_6, \quad C_{1,2} = M_4 + M_5,$$
$$C_{2,1} = M_6 + M_7, \quad C_{2,2} = M_2 - M_3 + M_5 - M_7.$$

§2.4 Der Algorithmus von Strassen

- Die Laufzeit dieses Verfahrens ergibt sich also als

$$T(n) = 7 \cdot T(n/2) + O(n^2).$$

- Master-Theorem: $a = 7, b = 2, f(n) = O(n^2)$.
- Fall 1: $T(n) = \Theta\left(n^{\log_2 7}\right) = O\left(n^{2.8074}\right)$.

Bemerkungen:

- Wie implementiert man den Algorithmus von Strassen, wenn n keine Zweierpotenz ist?
- Der Algorithmus von Strassen lohnt sich erst, wenn n ziemlich groß ist, etwa $n > 100$.
- …

§2.4 Der Algorithmus von Strassen

Bemerkungen:

- [1968]-[1990] viele kleine Verbesserungen …
- Coppersmith-Winograd [1990]: $O\left(n^{2.3755}\right)$.
- Stothers [2010]: $O\left(n^{2.3737}\right)$.
- Vassilevska Williams [2013]: $O\left(n^{2.3729}\right)$.
- Le Gall [2014]: $O\left(n^{2.3728639}\right)$.
- Alman, Williams [2020]: $O\left(n^{2.3728596}\right)$.
- Duan, Wu, Zhou [2022]*: $O\left(n^{2.371866}\right)$.
- Williams, Xu, Xu, Zhou [2024]*: $O\left(n^{2.371552}\right)$.
- Beste untere Schranke Raz [2002]: $\Omega\left(n^2 \log n\right)$

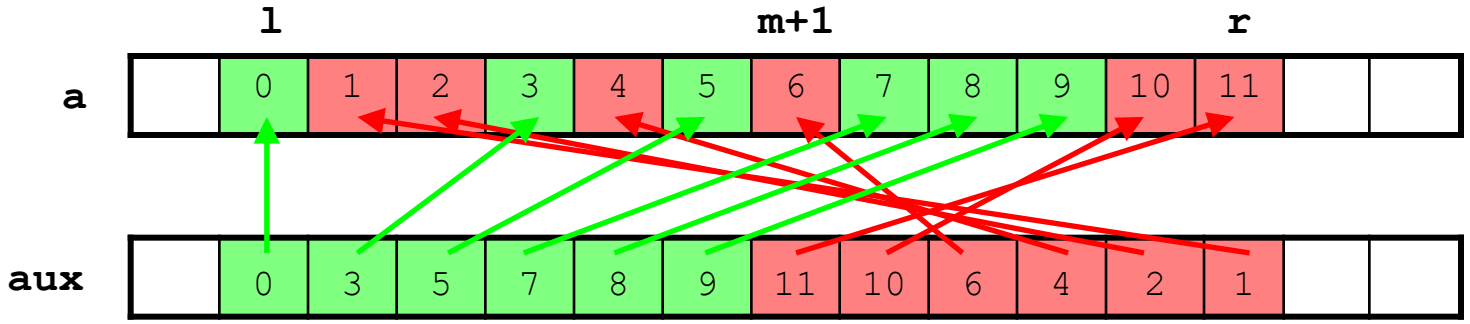
* under review

Merge-Sort

```
void itemArray::mergeSort(int left, int right)
{
   if (right<=left) return;
   int middle = (left+right)/2;
   mergeSort(left     , middle);
   mergeSort(middle+1, right );
   merge(left, middle, right);
}
```

Mischen (merge)

```
void itemArray::merge(int l, int m, int r)
{
   int i, j, k;
   for (i=m+1; i >l; i--)    aux[i-1]   = a[i-1];
   for (j=m  ; j <r; j++)    aux[r+m-j] = a[j+1];
   for (k=l  ; k=<r; k++)
      if (aux[j].key < aux[i].key) a[k] = aux[j--];
      else                         a[k] = aux[i++];
}
```



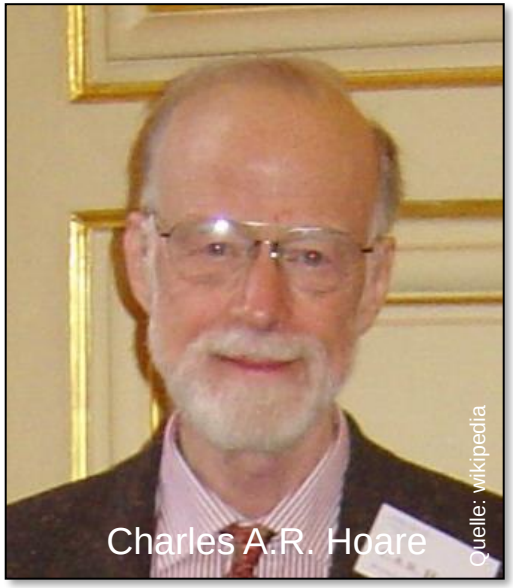
Laufzeit und Speicherbedarf

- Die Laufzeit von Merge-Sort hängt nicht von der Reihenfolge der Elemente bei der Eingabe ab.
- Die Laufzeit ist im best, average und worst case gleich.
- Anzahl der Vergleiche:

$$C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + n.$$

  - Master-Theorem mit $a = b = 2,\ f(n) = n.$
  - Fall 2a: $C(n) = O(n \log n)$.
- Anzahl der Vertauschungen analog: $M(n) = O(n \log n)$.
- Merge-Sort (in dieser Form) benötigt $O(n)$ zusätzlichen Speicher.
  - Mit Listen kann Merge-Sort auch in-situ implementiert werden.

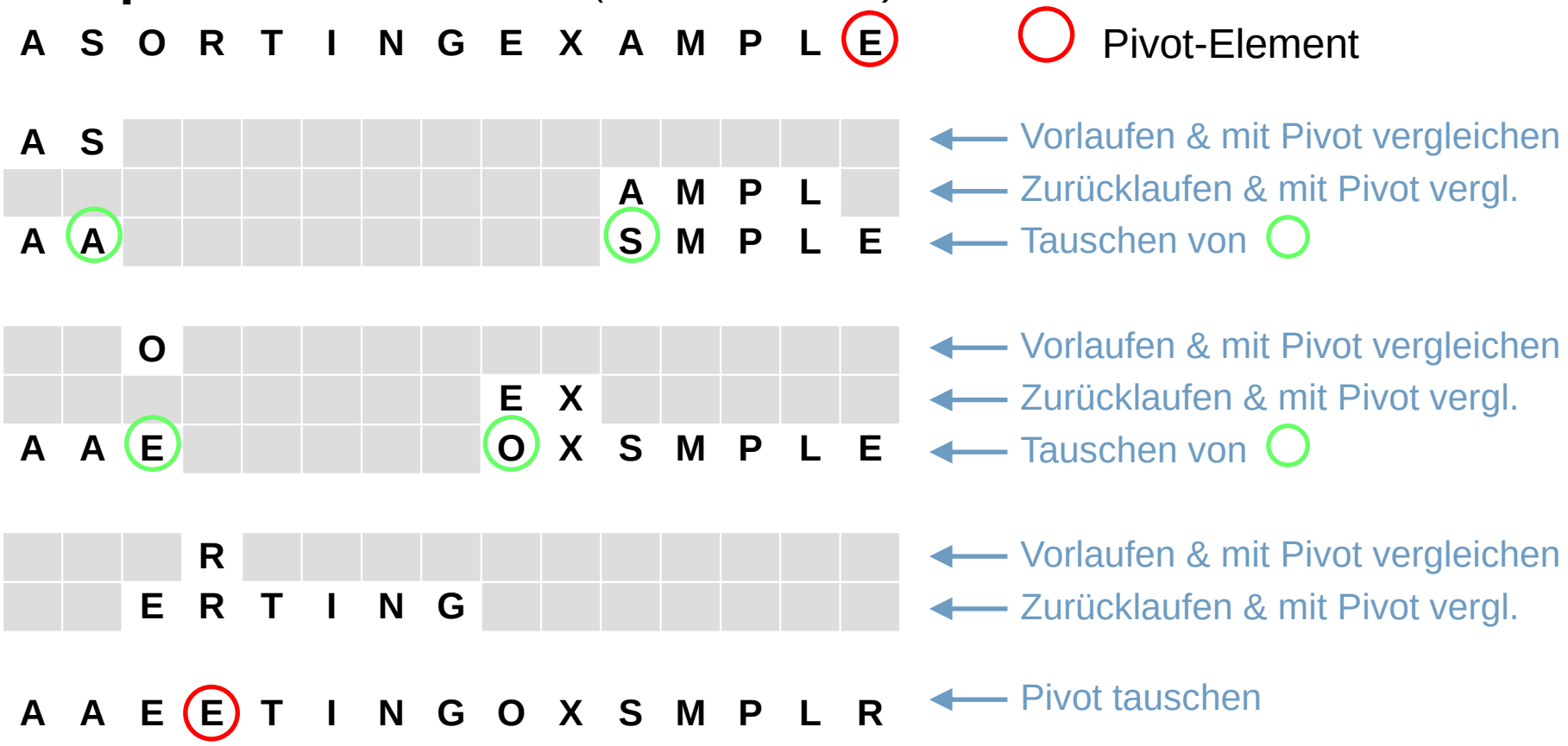
Quick-Sort: Idee (C.A.R. Hoare (1960))



- Teile-und-herrsche-Verfahren (divide-and-conquer)
  1. Aufteilung in Teilaufgaben (aka Partition/Zerlegung): Zerlege das Feld so, dass die folgenden drei Bedingungen erfüllt sind
     a) Das Element **`a[i]`** befindet sich für irgend ein **`i`** an seinem endgültigen Platz.
     b) Keines der Elemente in **`a[left],...,a[i-1]`** ist größer als **`a[i]`**.
     c) Keines der Elemente in **`a[i+1],...,a[right]`** ist kleiner als **`a[i]`**.

     Position der Zerlegung hängt von Reihenfolge der Elemente ab.
  2. Sortieren der Teile unabhängig voneinander: Wende Quick-Sort rekursive auf die Teilfelder an.

Beispiel: Partitionieren („Teile“-Schritt)

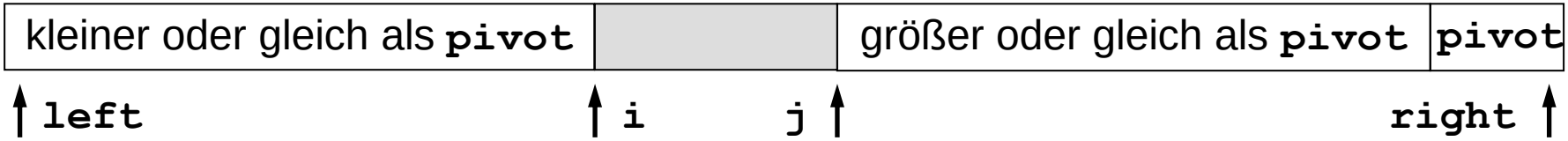


- Das Pivot-Element steht am Ende an seiner endgültigen Position!

Partitionieren

```
int itemArray::partition(int left, int right)
{
   int i = left-1, j = right;
   item pivot = a[right];
   for (;;) {
      while (a[++i].key < pivot.key) ;
      while (a[--j].key > pivot.key) if (j==left) break;
      if     (i>=j) break;
      swap(i,j);
   }
   swap(i,right);
   return i;
}
```

Übung



§2.5 Sortieralgorithmen

```
void itemArray::quickSort(int left, int right)
{
   if (right<=left) return;
   int i = partition(left, right);
   quickSort(left,i-1);
   quickSort(i+1,right);
}
```

Übung

Bemerkung:

- Der „Herrsche“-Schritt entfällt, weil für das Pivot-Element an der Stelle `i` die Bedingungen a) – c) erfüllt sind, d.h. es ist bereits an seiner endgültigen Position.

Laufzeit

- Die Anzahl der Schlüsselvergleiche in der Funktion **`partition`** ist kleiner als n.
- Best case:
  - Im Idealfall wird Menge der Elemente genau halbiert:

$$C(n) = 2\,C(n/2) + n.$$

  - Master-Theorem mit $a = b = 2,\ f(n) = n.$
  - Fall 2a: $C_{\min}(n) = O(n \log n).$

Übung

§2.5 Sortieralgorithmen

- Worst case:
  - Aufruf von Quicksort für n bereits sortierte Elemente:
  - Partitionen sind entartet.
  - Programm ruft sich selbst $n-$mal auf.
  - $C_{max}(n) = O(n^2)$,

    $M_{max}(n) = O(n^2)$.
- Average case: [Ottmann/Widmayer]
  - $C_{avg}(n) = O(n \log n)$
  - $M_{avg}(n) = O(n \log n)$

Übung

Weitere Quick-Sort-Varianten

- **Randomisiertes Quick-Sort:** Um auch für sortierte Elemente eine mittlere Laufzeit von $O(n \log n)$ zu bekommen, kann ein zufälliges Pivot-Element verwendet werden:
  1. Wähle ein zufälliges Element aus `a[left],...,a[right]` aus.
  2. Vertausche dieses mit `a[right]`.
  - Der Erwartungswert für die Laufzeit wird damit: $O(n \log n)$.
- **Hybrid-Algorithmus:** Kombination mit einem anderen Sortieralgorithmus, z.B. Heap-Sort, falls die Rekursionstiefe von Quick-Sort zu tief zu werden droht, z.B. $\geq 2\log n$.
  - Worst-case und average-case Laufzeit: $O(n \log n)$.
  - Intro-Sort (introspective sorting) [Musser97].

Komplexität des Sortierproblems?

- Gibt es (vergleichsbasierte, in-situ) Sortieralgorithmen, die im worst case eine bessere Laufzeit als $O(n \log n)$ haben?

- Das ist die Frage nach der Komplexität des Sortierproblems, wenn nur
  - Schlüsselvergleiche zwischen Elementen und
  - Elementvertauschungen

  als Operationen erlaubt sind!

- Wie viele Schlüsselvergleiche sind mindestens notwendig um n Elemente zu sortieren?

§2.5 Sortieralgorithmen

Was ist die Komplexität des Sortierproblems?

- Die Komplexität ist nach unten durch $\Omega(n \log n)$ beschränkt.

- Ist diese Schranke scharf?
- Es gibt einen Sortieralgorithmus, der die Laufzeit $O(n \log n)$ im worst case hat, z.B. Merge-Sort.

- Die Komplexität des Sortierproblems, das auf
  - Vergleichsoperationen zweier Elemente und
  - Vertauschungen von Elementen basiert,

  hat die Komplexität $\Theta(n \log n)$.

Bemerkung

Externe Sortierverfahren (ex-situ, engl.: out-of-place):

- **Radixsort** (Fachverteilen): Die Schlüssel sind Bit- oder Zeichenfolgen.
  1. Verteile Elemente gemäß der aktuellen Stelle im Schlüssel auf Fächer.
  2. Ordne die Elemente nach der Fachreihenfolge an.
  3. Dies wird von der letzten bis zur ersten Stelle im Schlüssel wiederholt.
  - Laufzeit, wenn die Schlüssel maximal d Stellen haben: $O(d \cdot n)$.
  - Diese Verfahren sind nicht vergleichsbasiert!

§2.5 Sortieralgorithmen

- **Beispiel Radixsort**: Sortiere $124, 523, 483, 128, 923, 584$
  - 3. Stelle:

0	1	2	3	4	5	6	7	8	9
			52**3** 48**3** 92**3**	12**4** 58**4**				12**8**	

    - $523, 483, 923, 124, 584, 128$
  - 2. Stelle:

0	1	2	3	4	5	6	7	8	9
		5**2**3 9**2**3 1**2**4 1**2**8						4**8**3 5**8**4	

    - $523, 923, 124, 128, 483, 584$
  - 1. Stelle:

0	1	2	3	4	5	6	7	8	9
	124 **1**28			**4**83	**5**23 **5**84				**9**23

    - $124, 128, 483, 523, 584, 923$

§2.6 Convex Hull

```
 1  SortedList SlowConvexHull(PointSet P)
 2  {
 3     PointSet E = ∅;
 4     for all (ordered pairs (p,q)∈P² with p≠q){
 5        valid = true;
 6        for all (points r∈P\{p,q}){
 7           if (r lies left of the ordered line pq)
 8              valid = false;
 9        }
10        if (valid) add the ordered line pq to E;
11     }
12     SortedList L = ConstructSortedList(E);
13     return L;
14  }
```

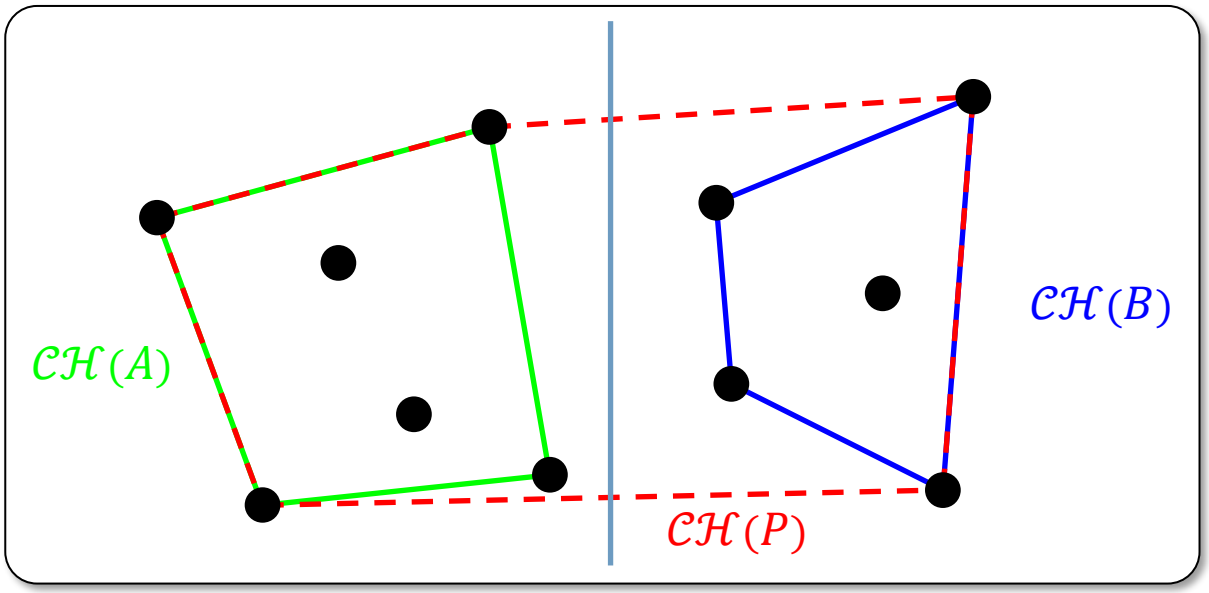
§2.6 Convex Hull

- Run time of the algorithm SlowConvexHull:
  - Line 4: $n^2 - n = n(n-1)$ pairs of points.
  - Line 6: $n - 2$ points are tested.
  - Line 7: To decide if a point lies left or right of a line is a primitive operation, so it is computable in constant time.
  - ▶ Lines 3-11: This yields a run time of $O(n^3)$.
  - Line 12: The brute force approach has run time $O(n^2)$.
- ▶ Total run time: $O(n^3) + O(n^2) = O(n^3)$.
- This can be improved!?

§2.6 Convex Hull

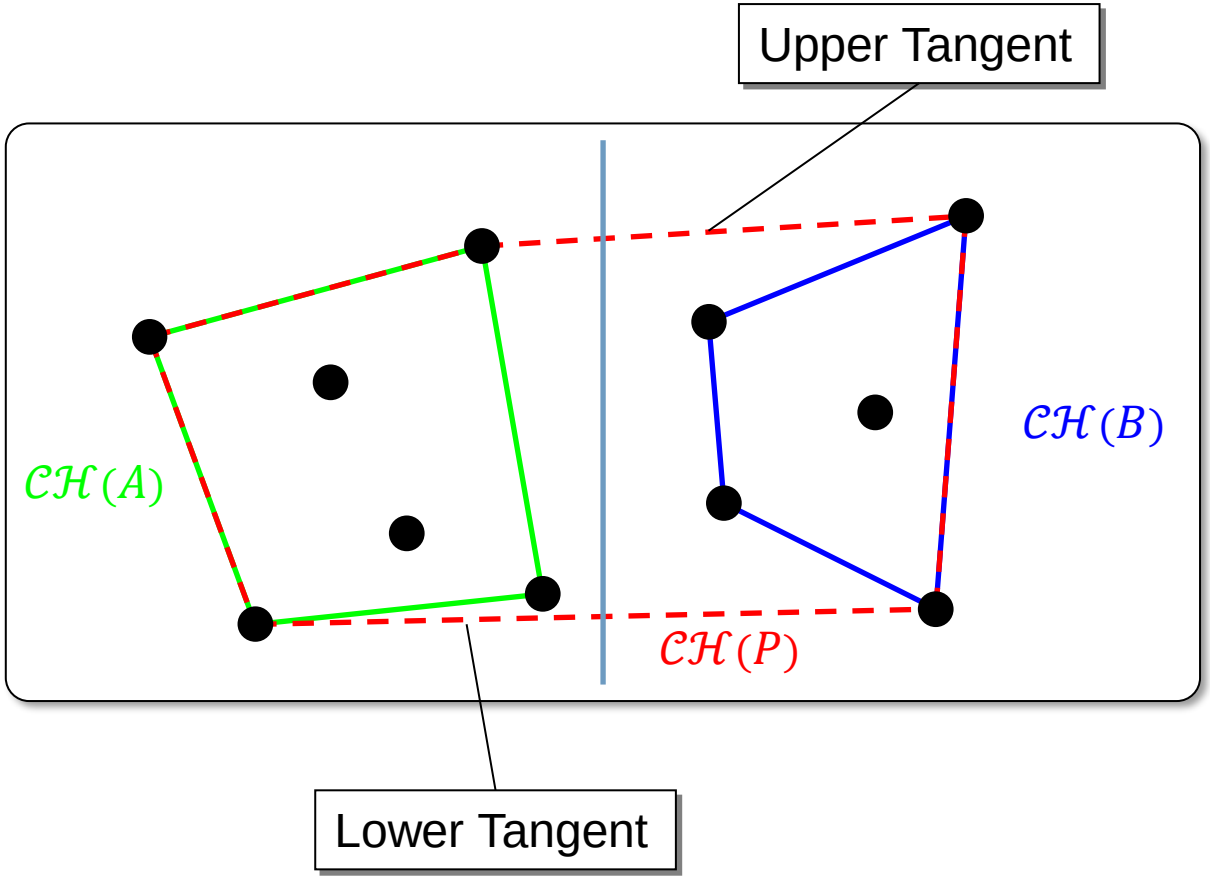
Divide-and-conquer-approach:

1. Divide point set P into two equal halves A and B.
   - Sort points by x-coordinate.
2. Recursively compute a left $\mathcal{CH}(A)$ and a right convex hull $\mathcal{CH}(B)$.
3. Merge $\mathcal{CH}(A)$ and $\mathcal{CH}(B)$ to obtain $\mathcal{CH}(P)$.



§2.6 Convex Hull

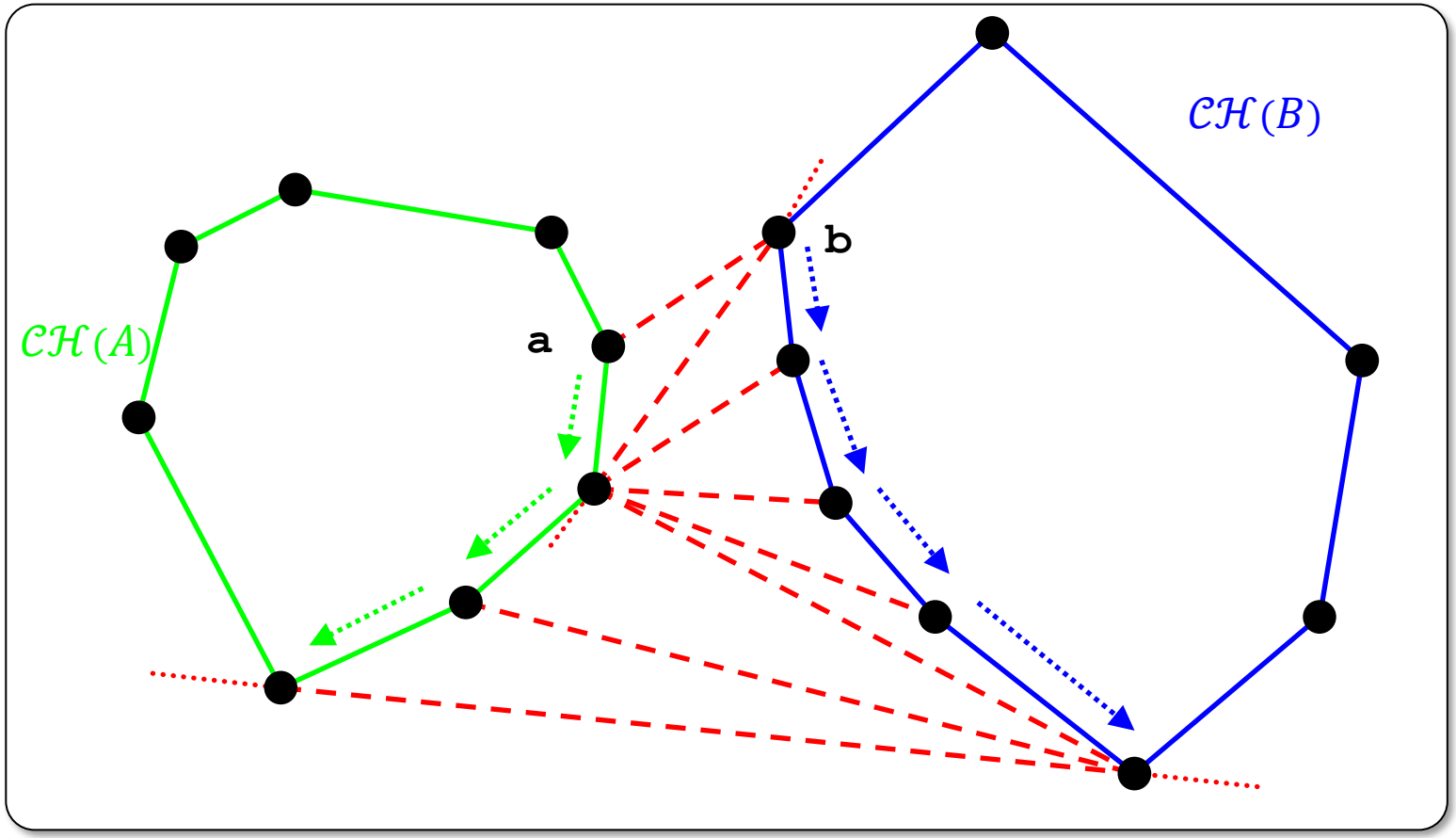
How to merge a left and a right convex sub-hull?



§2.6 Convex Hull

How to merge a left and a right convex sub-hull?

- Example: Lower Tangent



§2.6 Convex Hull

How to merge?

- Find the lower (and upper) tangent:

```
Line LowerTangent(ConvexHull CH(A), ConvexHull CH(B))
{
    Point a = RightMostPointOf(CH(A));
    Point b =  LeftMostPointOf(CH(B));
    while (ab is not lower tangent of CH(A) and CH(B)) {
       while (ab is not lower tangent of CH(A)) {
          a = a-1; // Move clockwise on CH(A)
       }
       while (ab is not lower tangent of CH(B)) {
          b = b+1; // Move counter-clockwise on CH(B)
       }
    }
    return ab;
}
```

§2.6 Convex Hull

Run time of this approach:

1. Divide point set P into two equal halves A and B.
- Sorting of n points takes $O(n \log n)$.
3. Merging: Computing tangents $O(n)$.
2. Recurrence: $T(n) = 2T(n/2) + O(n)$.
- Master-Theorem: $a = 2, b = 2, f(n) = O(n)$.

  Case 2: $T(n) = O(n \log n)$.

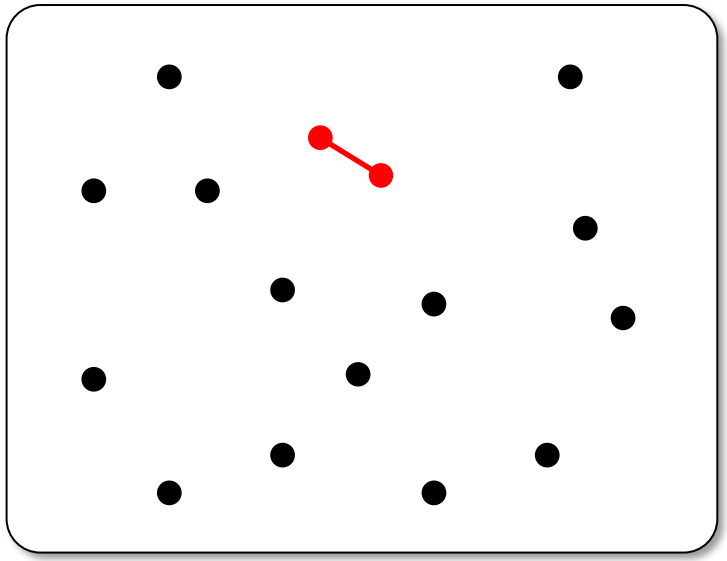
- Total run time: $O(n \log n)$.

§2.7 Berechnung des dichtesten Punktepaares

Gegeben: Eine Menge von Punkten $p_0, p_1, \ldots, p_{n-1}$.

Gesucht: Zwei Punkte mit dem geringsten (euklidischen) Abstand zueinander.

Beispiel: Menge von Punkten mit dichtestem Punktpaar in Rot.



§2.7 Berechnung des dichtesten Punktepaares

Algorithmus: Betrachte alle Punktpaare.

```
minDist = ∞;
for (int i=0; i<n-1; i++)
   for (int j=i+1; j<n; j++)
      if (dist(p_i, p_j) < minDist) minDist = dist(p_i, p_j);
```

Euklidischer Abstand: $\mathbf{dist(p,q)=\sqrt{(p.x-q.x)^2+(p.y-q.y)^2}}$

Analyse: $T(n) = O(n^2)$.

§2.7 Berechnung des dichtesten Punktepaares

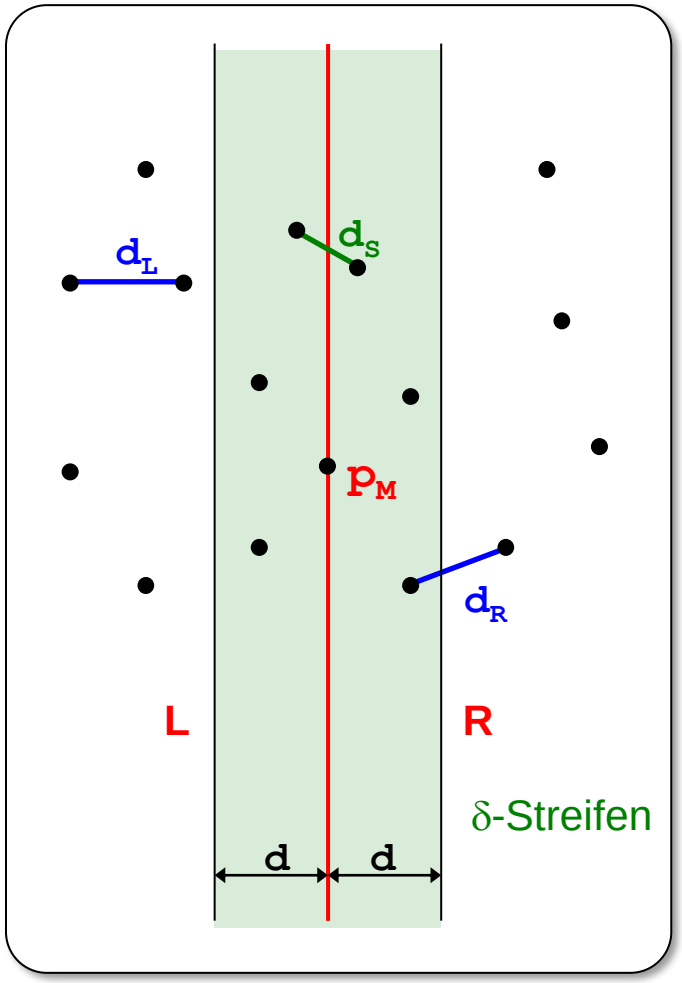
Idee:

(1) (a) Teile die Menge der Punkte längs der **x**-Achse in zwei Hälften L und R;
(b) Sei p_M der erste Punkt in R (Median);

(2) (a) Berechne minimale Distanz d_L in L;
(b) Berechne minimale Distanz d_R in R;
(c) `d = min(`d_L`,`d_R`);`

(3) δ-Streifen: Berechne für alle Punkte im Streifen um p_M mit der Breite von `2*d` die minimale Distanz d_S;

(4) `return min(d,`d_S`);`



§2.7 Berechnung des dichtesten Punktepaares

Analyse (prospektiv)

- Für (1) ist die einmalige Sortierung aller Punkte nach der $\mathbf{x}$-Achse besonders geschickt.
  - Die Sortierung kann in $O(n \log n)$ durchgeführt werden.
- Aufwand für die Berechnung von (2) – (4):
  - Behauptung: Die Berechnung der minimalen Distanz im δ-Streifen kann in $O(n)$ durchgeführt werden:

$$T(n) = 2 \cdot T(n/2) + O(n).$$

  - Master-Theorem: $a = 2,\ b = 2,\ f(n) = O(n)$.
  - ▶ Fall 2a: $O(n \log n)$.
- Gesamte Laufzeit von (1) – (4): $\Theta(n \log n)$.

§2.7 Berechnung des dichtesten Punktepaares

`mergeSort` zum Sortieren einer linear verketteten Liste von Punkten (1):

```
Node* mergeSort(Node* c, int n)
{
   if (n <= 1) return c;

   // Liste c in 2 Hälften a und b teilen:
   Node* a = c;
   Node* q = c;
   for (int i=1; i<n/2; i++) q = q->next;
   Node* b = q->next;
   q->next = 0;

   // Liste a und b sortieren und verschmelzen:
   return merge(mergeSort(a,n/2), mergeSort(b,n-n/2));
}
```

```
struct Point
{
   double x, y;
};
```

```
struct Node
{
   Point pt;
   Node* next;
};
```

§2.7 Berechnung des dichtesten Punktepaares

`mergeSort` zum Sortieren einer linear verketteten Liste von Punkten (1):

```
Node* merge(Node* a, Node* b)
{
   Node *c = 0, *t = 0;
   do {
      if (less(a,b)) {
         if (t == 0) c      =a;
         else        t->next=a;
         t = a; a = a->next;
      } else {
         if (t == 0) c      =b;
         else        t->next=b;
         t = b; b = b->next;
      }
   } while (a != 0 || b != 0);
   return c;
}
```

```
bool less(Node* a, Node* b)
{
   if (a==0) return false;
   if (b==0) return true;
   if (pass==1)
      return a->pt.x<b->pt.x;
   else
      return a->pt.y<b->pt.y;
}
```

Über die globale Variable **`pass`** wird gesteuert, ob nach der **`x`**- oder der **`y`**-Koordinate sortiert wird.

§2.7 Berechnung des dichtesten Punktepaares

Bestimmung der kürzesten Distanz im δ-Streifen:

- Es genügt die letzten 5 aufeinander folgenden Punkten im δ-Streifen **u, v, w, z** und **p** zu betrachten, um den kleinsten Distanzwert zu berechnen.

```
for (int i=1; i<=n; i++)
{
   p = p_i;

   if (p liegt im 2*d breiten δ-Streifen)
   {
      Seien u,v,w,z die vier letzten Punkte,
         die im δ-Streifen lagen;
      d = min(d, dist(u, p));
      d = min(d, dist(v, p));
      d = min(d, dist(w, p));
      d = min(d, dist(z, p));
   }
}
```

$p_1, p_2, \ldots, p_n$ seien alle **n** Punkte nach der **y**-Koordinate sortiert.

Der δ-Streifen wird schmaler, sobald die Minimumsbildung einen kleineren Distanzwert **d** liefert.

§2.7 Berechnung des dichtesten Punktepaares

Kompletter Algorithmus:

```
Point  p1Min, p2Min;
double minDist;
int    pass;

void closestDistance(Node* pl, int n)
{
   p1Min   = 1. Punkt aus der Liste pl;
   p2Min   = 2. Punkt aus der Liste pl;
   minDist = dist(p1Min, p2Min):

   pass = 1; pl = sort(pl,n);
   pass = 2; pl = sort(pl,n);

   p1Min, p2Min, minDist ausgeben;
}
```

p1Min, p2Min, minDist sind globale Variablen zur Speicherung der aktuellen minimalen Distanz.

pl ist eine beliebige linear verkettete Liste mit **n** Punkten.

Sortierung nach der **x**-Koordinate.

Sortierung nach der **y**-Koordinate und Bestimmung der kürzesten Distanz werden simultan durchgeführt!

§2.7 Berechnung des dichtesten Punktepaares

Kompletter Algorithmus:

```
Node* sort(Node* c, int n)
{
   if (n <= 1) return c;
   // Liste c in 2 Haelften a und b teilen:
   Node* a = c;
   Node* q = c;
   for (int i=1; i<n/2; i++) q = q->next;
   Node* b = q->next;
   q->next = 0;
   // x-Koordinate für δ-Streifen:
   double midX;
   if (pass == 2) midX = b->pt.x;
   // Liste a und b sortieren und verschmelzen:
   c = merge(mergeSort(a,n/2), mergeSort(b,n-n/2));
   ...
}
```

§2.7 Berechnung des dichtesten Punktepaares

```
Node* sort(Node* c, int n)
{
   ...
   // Bestimmung der kürzesten Distanz im δ-Streifen
   if (pass == 2) {
      Point *u=0, *v=0, *w=0, *z=0;
      for (q = c; q != 0; q = q->next) {
         Point* p = &(q->pt);
         if ( abs(p->x - midX) < minDist ) {
            updateMinPair(p,u);
            updateMinPair(p,v);
            updateMinPair(p,w);
            updateMinPair(p,z);
            u = v; v = w; w = z; z = p;
         }
      }
   }
   return c;
}
```

§2.7 Berechnung des dichtesten Punktepaares

```
void updateMinPair(Point* p1, Point* p2)
{
   if (p1 == 0 || p2 == 0) return;

   double d = dist(*p1,*p2);
   if (d < minDist)
   {
      minDist = d;
      p1Min   = *p1;
      p2Min   = *p2;
   }
}
```

Analyse (retrospektiv)

- Die Routine **`closestDistance`** ruft zweimal **`sort`** auf, um erst nach der **`x`**- und dann nach der **`y`**-Koordinate zu sortieren.
- Der erste Aufruf entspricht einem einfachen Aufruf von Mergesort.
  - Dieser Aufruf hat Laufzeit $\Theta(n \log n)$.
- Der zweite Aufruf entspricht einem Aufruf von Mergesort mit anschließendem Durchlauf durch den δ-Streifen.
  - Aufwand: $T(n) = 2 \cdot T(n/2) + O(n) + O(n)$.
  - Master-Theorem: $\Theta(n \log n)$.
- Gesamter Aufwand: $\Theta(n \log n)$.

